Pandas and Polars

Performance, Memory Efficiency and Usability in Python Data Manipulation

Prepared by: M Yogeswar

1.Pandas and Polars?

Pandas is a widely used Python library for working with structured data. It provides DataFrames and Series for cleaning, filtering, joining, grouping, summarizing and analyzing data. It integrates with many Python data-science tools and is often convenient for exploratory data analysis and general-purpose analytics.

Polars is a DataFrame library built on a Rust-based execution engine. It supports column-oriented data processing, parallel execution, and both eager and lazy workflows. Lazy execution can optimize a sequence of operations before running them. Polars is designed for efficient data transformations, especially in larger or more complex processing pipelines.

2. Performance, Memory Efficiency and Usability Comparison

Aspect	Pandas	Polars
Performance	Mature and versatile; speed varies by operation, data type and dataset size.	Can benefit from parallel execution and lazy query optimization; not faster for every operation.
Memory efficiency	Can use more memory, particularly for object/string columns.	Can store data more compactly; actual usage depends on data types and workload.
Usability	Familiar API and broad integration with Python data-science tools.	Expression-based API with eager and lazy workflows; may require learning different patterns.
Suitable use cases	Exploratory analysis, statistics, notebooks and integration with ML libraries.	Efficient transformations, larger tabular workloads and pipelines that benefit from optimization.

3. Environment Setup in Google Colab

The experiment was carried out in Google Colab, a hosted Python notebook environment. Pandas, Polars, NumPy and Matplotlib were imported or installed. The Online Retail data was downloaded and prepared as a common CSV file, and both libraries loaded the same sample to keep the comparison consistent.

Tools used: Python, Google Colab, Pandas, Polars, NumPy and Matplotlib.

4. Dataset Selection

The selected dataset was the **Online Retail dataset** from the UCI Machine Learning Repository. It contains transaction-level information such as invoice numbers, product codes, descriptions, quantities, invoice dates, unit prices, customer IDs and countries. A sample of **150,000 records** was used to keep the experiment manageable while providing a useful data-processing workload.

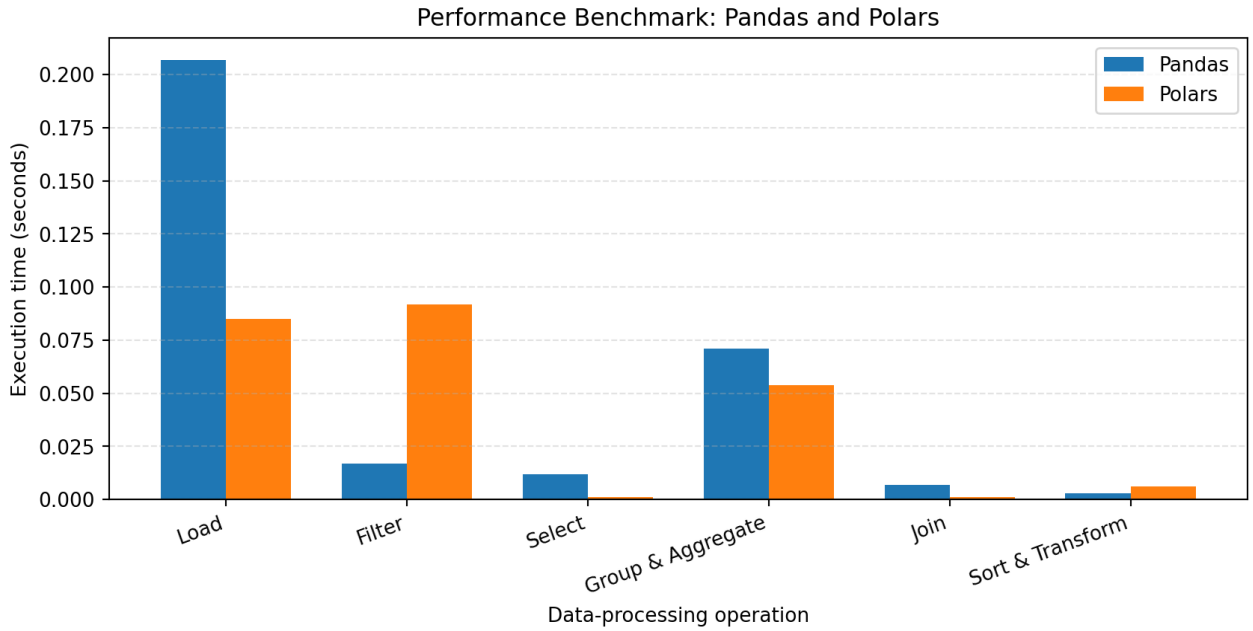
5. Implementation Methodology

Equivalent operations were performed in Pandas and Polars using the same CSV sample. Execution time was measured with Python's high-resolution timer. The resulting tables were inspected and aggregated revenue values were checked for agreement. The workflow was:

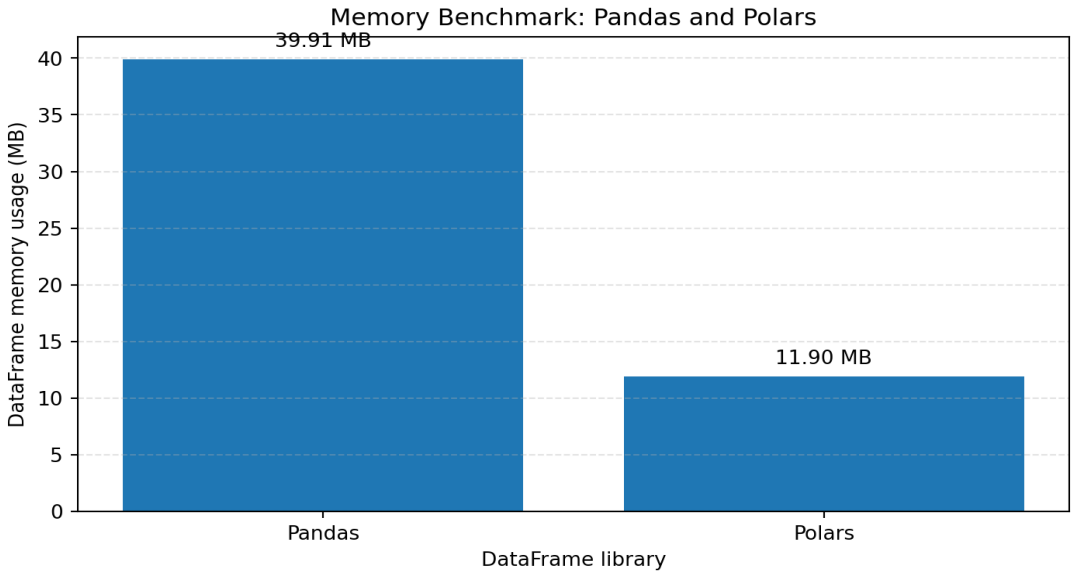
1. **Load:** read the common CSV file into both DataFrame libraries.
2. **Filter rows:** retain records with positive quantity and unit price.
3. **Select columns:** keep the fields required for sales analysis.
4. **Group and aggregate:** calculate total quantity, revenue and unique invoice counts by product code.
5. **Join:** attach product descriptions from a product lookup table.
6. **Sort and transform:** sort products by revenue and calculate revenue per unit and revenue share percentage.

6. Performance and Memory Benchmark Analysis

The following diagrams present the benchmark results captured in the supplied screenshots. The operation timings below are approximate readings from the performance chart, while the memory values are transcribed from the labels in the memory chart. Timings may vary across repeated runs and environments.



Performance results: The chart suggests Polars loaded the dataset faster (about 0.085 s versus 0.207 s) and was faster for selection, grouping/aggregation and joining in this run. Pandas filtered the rows faster (about 0.017 s versus 0.092 s). The sort-and-transform timings were both small, with Pandas appearing faster in the displayed run. These results show that the faster library can differ by operation.



Memory results: Pandas used 39.91 MB and Polars used 11.90 MB for the loaded DataFrames in the displayed measurement. Polars therefore used approximately 70.2% less DataFrame memory in this experiment. This metric represents the DataFrame footprint, not peak process memory during execution.

7. Key Observations

• Both libraries completed the required data-manipulation workflow and produced matching aggregate revenue values.
• Polars used substantially less DataFrame memory in the recorded measurement.
• Polars was faster for several operations, but Pandas was faster for filtering in this particular run.
• Results depend on data types, library versions, runtime conditions and execution mode. Single-run measurements are useful for initial comparison but are not universal performance guarantees.

8. Analysis and Conclusion

Strengths of Pandas: Pandas has a familiar and flexible API, a mature ecosystem, extensive documentation and strong integration with many statistics, visualization and machine-learning tools. It is particularly suitable for exploratory data analysis, small-to-medium datasets and workflows that rely on other Python libraries.

Limitations of Pandas: DataFrames containing many object or string columns can consume substantial memory. Some operations can become slower as datasets grow, and standard Pandas operations do not automatically parallelize every task across CPU cores.

Strengths of Polars: Polars supports parallel processing, a columnar execution engine and lazy query optimization. In the results shown here, it had a smaller DataFrame memory footprint and performed better on several measured operations. It is a strong option for efficient transformation pipelines and workloads that benefit from optimized execution.

Limitations of Polars: Its API differs from Pandas, so users may need to learn new expression and lazy-execution patterns. Some existing workflows and third-party tools are more directly integrated with Pandas, and Polars is not guaranteed to outperform Pandas for every operation or dataset.

Conclusion: This experiment compared Pandas and Polars using 150,000 Online Retail transaction records and the same sequence of loading, filtering, selecting, grouping, joining, sorting and transforming operations. The visual results show that Polars used less DataFrame memory and was faster for several operations, while Pandas performed better for filtering in the displayed run. Pandas is a practical choice for general data analysis and broad ecosystem compatibility; Polars is a compelling choice for memory-conscious and optimized data-processing pipelines. The best library should be chosen based on measured performance, memory requirements, usability and project needs.